ATLM

iscte INSTITUTO UNIVERSITÁRIO DE LISBOA

Advanced Techniques for Language Models

Mini-Assignment 2

António Cruz (140129), Bruno Santos (140586), Pedro Miranda (129268)

Index

1 Introduction

Mini-Assignment 1 produced SmolLM2-360M [1], a decoder model adapted to the IT job-postings domain by continued next-token pretraining on raw Djinni descriptions. The resulting checkpoint is a domain-fluent text completer: given a partial posting it can continue it, but it cannot take a sentence-length recruiter request and produce a complete, structured job posting from it. Mini-Assignment 2 is the alignment stage that adds that capability.

Alignment proceeds in two sub-stages, both implemented in the accompanying notebook and covered by this report. **Supervised fine-tuning** (SFT) teaches the model to recognise a recruiter request and produce a structured response by training on roughly 7,500 (query, posting) pairs. **Preference alignment** via Direct Preference Optimisation [4] (DPO) then refines those responses against a written rubric, faithfulness to the request, four required Markdown sections, professional language, absence of repetition, using preferences ranked by a separate local LLM acting as judge, a setup known as Reinforcement Learning from AI Feedback [2] (RLAIF).

Three external LLMs participate across the pipeline, assigned to non-overlapping roles to avoid shared-prior contamination. Gemma 4 E2B acts as the ETL teacher that generated the SFT corpus and plays no judge role. Nemotron Nano 4B is the RLAIF listwise judge that produced the preference triples. Granite 3.3 2B is the pairwise evaluation judge. Both judge models were selected from a six-candidate calibration battery; the selection rationale is detailed in Section 4.

The report walks the full pipeline and ends with a six-way comparison on a fixed set of twenty held-out recruiter prompts: the untrained base, the SFT model, and four DPO-aligned variants at $\beta \in \{0.05, 0.10, 0.20, 0.30\}$. The deliverable policy is DPO-β=0.10. Section 7 documents the failure modes discovered during execution, including an inference-time regression that produced catastrophic token-loop collapses, and the structural discrimination ceiling of the evaluation judge on close-pair comparisons.

1.1 Pipeline Overview

Table 1 summarises the four training stages. Mini-Assignment 2 covers the last two.

Stage	Name	Capability added	Assignment
1	Pretraining	General language understanding	(authors of SmolLM2)
2	Continued pretraining	Domain fluency on IT job postings	Mini-Assignment 1
3	Supervised fine-tuning	Recruiter instruction following	Mini-Assignment 2
4	Preference alignment	Rubric-consistent output quality	Mini-Assignment 2

Table 1: Four-stage post-training pipeline. Each stage adds a distinct capability on top of the previous checkpoint.

2 Starting Point and SFT Data Preparation

2.1 The Mini-Assignment 1 Checkpoint

Mini-Assignment 1 delivered two variants of SmolLM2-360M: a full fine-tune and a LoRA adapter [3]. The LoRA variant was selected as the best in-domain fit and is the checkpoint carried forward here.

Before any further training, the adapter is merged into the base with `merge_and_unload`. Merging produces one consolidated checkpoint equivalent at inference time to (base + Mini-Assignment 1 LoRA), but with no PEFT bookkeeping. Every subsequent stage trains a fresh LoRA on top of a plain base, so there is no stacked-adapter arithmetic and no question of which adapter is active. The cost is a one-off 694 MB checkpoint on disk. From this point on the notebook treats the merged model as its pretraining starting point.

2.2 SFT Corpus Generation

A supervised fine-tuning dataset does not exist for this domain. It was generated by an ETL agent, `atlm_teacher`, a Gemma 4 E2B model served locally via `agent_server`. Given a raw Djinni posting the agent produces three short recruiter queries and one clean structured job description in Markdown (`## Summary`, `## Required Skills`, `## Responsibilities`, `## Requirements`), or a skip marker for unusable input. The output contract was validated on a diverse sample before the bulk run.

The bulk ETL converted 2,507 raw postings. Each record carries three independent recruiter queries pointing to the same job description; fanning these out yields 7,521 (query, posting) training pairs. Reuse of the same posting target under three different phrasings is a cheap augmentation that teaches the model phrasing-robustness without any extra annotation cost.

The records are split 90/10 into train and validation *at the record level*, not at the query level, so no job description leaks across the split. With seed 42, this produces 2,257 training records (6,771 examples) and 250 validation records (750 examples). The split is also re-used in Section 4: the 250 held-out validation records become the preference prompts, keeping the preference data strictly separate from SFT training without requiring additional annotation.

2.3 Prompt Template

SmolLM2-360M has never been instruction-tuned; it has no notion of a request versus a response boundary. A consistent prompt template provides that signal during SFT and is reused unchanged at inference. The template is Alpaca-inspired:

```
You are a recruitment assistant. Given a brief recruiter
request, write a complete structured job posting in Markdown.

### Request
```

```
{query}


### Posting
{jd}
```

At inference time the model receives everything up to and including `### Posting\n` and generates the posting from there. Three design decisions governed the choice over leaner key-value and ChatML alternatives:

1. **Explicit task framing.** The one-sentence preamble costs roughly 25 tokens and gives the model an unambiguous anchor for the task.
2. **No heading clash.** The structured postings already use `##` headings; using `###` for template separators lets the model distinguish template markers from response headings.
3. **No special-token machinery.** Plain-text separators tokenise into existing vocabulary, so no new embeddings are added and no chat template needs registering on the tokenizer.

3 Supervised Fine-Tuning

3.1 Rationale

DPO and other preference-alignment methods assume the policy already follows instructions: they refine the *quality* of responses, not the *format*. A domain-adapted text completer does not satisfy that precondition. Supervised fine-tuning bridges the gap by training the model explicitly on (request, response) pairs so it learns where a recruiter prompt ends and where a structured posting should begin.

3.2 Training Configuration

A fresh LoRA adapter is trained on top of the merged Mini-Assignment 1 checkpoint, leaving the 361.8M base weights frozen. The LoRA shape matches the Mini-Assignment 1 adapter: rank 16, scaling factor 32, dropout 0.05, applied to all seven attention and feed-forward projection matrices (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`), so the alignment delta at the next stage is comparable. Table 2 summarises the full configuration.

Two epochs is deliberately short. The goal of this stage is to teach the instruction-following format, not to re-adapt to the domain; over-training here would erode the domain knowledge acquired in Mini-Assignment 1. A maximum sequence length of 1,024 tokens covers every (query, posting) pair without truncation and matches the tokenisation cap used in Mini-Assignment 1.

The loss is computed over the entire formatted sequence: preamble, request field, and response field; rather than only the response tokens. Masking the prompt span would be the theoretically cleaner choice; for a model of this size the difference is minor and the simpler configuration was retained.

Hyperparameter	**Value**
LoRA	
Rank r	16
Scaling α	32
Dropout	0.05
Target modules	all attention + FFN projections (7 matrices)
Training	
Epochs	2
Learning rate	2×10^{-4}
Effective batch size	16 (micro-batch 4 × gradient accumulation 4)
Max sequence length	1,024 tokens
Precision	bf16
Warmup ratio	0.03
Weight decay	0.01
Seed	42
Framework	HuggingFace TRL 1.5.1 `SFTTrainer`

Table 2: Supervised fine-tuning configuration.

3.3 Results

The run completed in 17.4 minutes on a single RTX 4090 (24 GB). Table 3 records the key outcomes.

Metric	**Value**
Trainable parameters (LoRA)	8,683,520
Trainable / total parameters	2.34% of 370.5M
Training examples	6,771
Validation examples	750
Final validation loss	1.4955
Final validation perplexity	4.46
Wall-clock time	17.4 min (RTX 4090)

Table 3: Supervised fine-tuning outcomes.

The 750 validation examples correspond to the 250 held-out records times three queries, confirming that the record-level split from Section 2.2 held correctly.

3.4 Sanity Check

A qualitative check runs the same recruiter prompt through both the merged Mini-Assignment 1 model and the SFT model with greedy decoding. The pre-SFT model treats the prompt as a free-continuation task: it begins with loosely relevant sentences and quickly degrades into hallucinated lists of unrelated occupations. The SFT model recognises the prompt as an instruction, emits a title and the four required Markdown sections (`## Summary`, `## Required Skills`, `## Responsibilities`, `## Requirements`), and stays on topic throughout. Figure 1 illustrates this contrast on the Senior Data Scientist prompt.

The full six-way comparison across all model states is deferred to Section 6, where the same fixed prompt set and order-swap protocol are applied to all models simultaneously.

MA1 only - text completer

The post is open to all candidates who have at least 2 years experience...

Requirements:
– Experience working as an Assistant;
– Basic knowledge of statistics...

[degrades into unrelated occupations:]

...massage therapist, acupuncturist, hypnotist, yoga teacher, martial arts instructors...

MA1 + SFT - instruction follower

```
# Junior/Senior ML Engineer
## Summary
```
This role involves working closely with data scientists...
```
## Required Skills
```
– Advanced proficiency in Python...
```
## Responsibilities
```
– Developing new features...
```
## Requirements
```
– 3+ years of experience...

Figure 1: Sanity check after SFT. Left: the MA1 model continues text freely and degrades into hallucinated content. Right: the SFT model recognises the prompt as an instruction and produces the required Markdown structure. Same prompt, greedy decoding.

4 Preference Dataset Construction

Direct Preference Optimisation requires a dataset of (prompt, chosen, rejected) triples. Rather than use a generic preference dataset or annotate by hand, the project builds its own through Reinforcement Learning from AI Feedback (RLAIF) [2]: candidates are sampled from the SFT model and ranked by a separate, stronger LLM. Three design decisions (prompt source, candidate sampling, and judge selection) are detailed below.

4.1 Preference Prompts

The 250 preference prompts come from the SFT validation hold-out introduced in Section 2.2: one query per record, drawn with a separate seed so the pick does not correlate with the record shuffle. This choice satisfies two constraints simultaneously. The SFT model has never trained on these prompts, so the sampled candidates reflect generalisation rather than memorisation. The 250 records are also kept strictly separate from the 20 frozen evaluation prompts used in Section 6, which are hand-written and cannot appear in `converted.jsonl` by construction.

4.2 Candidate Sampling

Four candidates are sampled from the SFT model per prompt using temperature 0.9 and nucleus sampling ($p = 0.95$), with `num_return_sequences=4` in a single batched `generate` call and a maximum of 500 new tokens. Stochastic decoding is required: greedy decoding would produce near-identical samples across the four slots, leaving the judge nothing to rank. The four-candidate budget was chosen as the smallest set that gives a full-ranking judge six distinct ordered pairs per prompt (see Section 4.4).

4.3 Cross-Judge Calibration

Choosing the judge model required evidence rather than a guess. A calibration battery probed all five non-Gemma chat models available on `agent_server`: Nemotron Nano 4B, Granite 3.3 2B,

Qwen3.5, Ministral, and SmolLM3; plus Gemma 4 E2B as a control, on small fixed samples of the actual MA2 judging tasks: 5 listwise RLAIF prompts and 5 pairwise evaluation pairs (each pair judged in both AB and BA order, 10 calls per model on the eval task). Gemma was excluded from the two MA2 judge roles regardless of its calibration score, because it also served as the ETL teacher that generated the SFT corpus; reusing it as judge would introduce shared-prior contamination across three pipeline roles.

Disqualifiers applied strictly: RLAIF parse rate below 9/10, eval AB-BA consistency below 7/10, or failure to detect the `ood-03` collapse case (DPO-b01 producing a 51-item repetition loop on an Elixir prompt, included as a deliberate hard sanity check). Table 4 reports the full results.

Model	RLAIF parse	Reasoning	RLAIF budget	Eval consistent	ood-03	Eval budget
`gemma-4` (control)	10/10	yes	8.9 min	9/10		6.1 min
`nemotron`	**5/5**	yes	5.4 min	4/5		3.1 min
`granite-3.3`	1/5	no	1.8 min	**4/5**		**1.4 min**
`smollm3`	5/5	no	1.6 min	3/5	×	0.9 min
`qwen3.5`	1/5	yes (capped)	91 min	0/5	×	31 min
`ministral`	5/5	yes (bimodal)	48.8 min	2/5		3.2 min

Table 4: Cross-judge calibration battery results. Bold entries are the passing scores on each metric. `nemotron` is assigned the RLAIF role; `granite-3.3` is assigned the eval role.

Nemotron is assigned the RLAIF role: the only non-Gemma candidate with 100% listwise parse rate, genuine chain-of-thought reasoning (4,898 characters on average, comparable to Gemma's 4,276), moderate Gemma disagreement (cross-judge signal is the goal), and a tractable 5.4-minute budget. Granite 3.3 is assigned the evaluation role: it matches Nemotron on every quality metric on the pairwise task, is faster (1.4 vs. 3.1 minutes), and belongs to a different model family, preserving cross-judge independence between the two roles. The limitation that this 10-call calibration probe did not surface: a structural position-bias ceiling on close pairs at production scale is the subject of Section 7.

4.4 RLAIF Judging

The judge receives a recruiter request and four numbered candidate postings. It applies the rubric in priority order: faithfulness to the request, structure and completeness (all four Markdown sections present and non-empty), professional language, and absence of repetition or truncation; and ends its response with a single parseable line:

```
RANKING: <best> > <2nd> > <3rd> > <worst>
```

This full-ranking format produces six (higher, lower) pairs per prompt, multiplying the preference signal volume by six versus the best-worst variant used in earlier configurations (one pair per prompt). The judge runs deterministically (`temperature=0.0`, `max_tokens=16384`, chain-of-thought enabled via `enable_thinking=true`) through the `atlm_rlaif_judge` preset registered server-side on `agent_server`, so the rubric is auditable independently of the notebook code.

Of 250 prompts, 241 were ranked successfully (9 parse failures), yielding **1,446 preference triples**. This is a 6.5× expansion over the 222 best-worst triples produced in prior configurations on the same prompt set. Whether the volume increase translates to a DPO quality improvement is tested in Sections 5 and 7.

5 DPO Training

5.1 Method and Starting Point

Direct Preference Optimisation [4] (DPO) updates the policy so that, on the same prompt, it assigns higher log-probability to the chosen response than to the rejected one, regularised by KL divergence to a frozen reference. Compared with PPO-based RLHF it requires no separate reward model and no rollout loop, making it tractable on a single consumer GPU. TRL 1.5.1's `DPOTrainer` is used throughout.

Following the same merge-between-stages pattern used at the Mini-Assignment 1 to SFT boundary, the SFT LoRA is first merged into the base with `merge_and_unload`, producing a consolidated checkpoint at `outputs/ma2-360m-sft-merged/`. A fresh LoRA is then trained on top for each DPO run. This keeps each stage's delta independent and avoids stacked-adapter arithmetic. The merged SFT checkpoint serves as both the policy initialisation and, implicitly, the reference: TRL computes reference log-probabilities by temporarily disabling the active adapter rather than loading a second model copy, saving a full model's worth of VRAM.

Four DPO runs launch sequentially from a single idempotent training cell at $\beta \in \{0.05, 0.10, 0.20, 0.30\}$, all other hyperparameters held constant. Beta is the KL coefficient: lower values allow the policy to diverge further from the SFT reference (stronger preference learning); higher values keep it conservative. The fourth point at $\beta = 0.05$ was added as a deliberate cliff probe after a three-point sweep (0.10, 0.20, 0.30) returned a monotonically lower-is-better result that could not distinguish a true trend from a peak within the tested range.

5.2 Training Configuration

Table 5 summarises the configuration shared across all four runs.

The effective batch size is halved relative to SFT (8 vs. 16) because DPO performs a forward pass through both the chosen and rejected responses at every step, roughly doubling the memory footprint per batch. Five epochs over 1,302 triples was determined empirically: fewer epochs left reward margins near zero; more caused the train reward accuracy to saturate at 1.00 with no corresponding deployment-side benefit (see Section 7).

5.3 Learning Rate Sensitivity

The learning rate required iteration. Three concrete values were tested at fixed $\beta = 0.10$:

Hyperparameter	**Value**
DPO-specific	
Beta values (β)	0.05, 0.10, 0.20, 0.30
Training triples	1,302 (90% of 1,446; 144 held out for eval)
LoRA	
Rank r	16
Scaling α	32
Dropout	0.05
Target modules	all attention + FFN projections (7 matrices)
Training	
Epochs	5
Learning rate	5×10^{-5}
Effective batch size	8 (micro-batch 2 $\times$ gradient accumulation 4)
Max sequence length	1,024 tokens
Precision	bf16
Seed	42
Framework	HuggingFace TRL 1.5.1 `DPOTrainer`

Table 5: DPO training configuration, shared across all four beta runs.

- 5×10^{-6}, the textbook DPO default for full fine-tuning: near-zero reward margins across all betas. The policy did not move meaningfully within the LoRA subspace.
- 5×10^{-5} (deliverable): clean positive margins, monotonic improvement in eval reward accuracy across the in-range betas, no fluency degradation. Section 6 confirms the same direction on the deployment-side win-rate.
- 1×10^{-4} (aborted): higher training-side reward margin than 5×10^{-5}, but visible fluency collapse on the held-out evaluation: including a hallucinated phrase appearing across multiple unrelated prompts and perplexity rising from 4.64 to 8.15.

The interpretation is that LoRA-DPO updates a small subspace of the full parameter space and requires a proportionally higher learning rate to move the reward function; but above 5×10^{-5} the gradient signal from 1,446 triples is too concentrated in too few parameters and forces fluency collapse. The combination ($\beta = 0.10,\ \text{lr} = 5 \times 10^{-5}$) is the hyperparameter answer this work supports for LoRA-DPO on SmolLM2-360M with this preference set.

5.4 Training-Side Results

Table 6 reports the held-out preference validation metrics for the four runs. Each run took approximately 30 minutes on the RTX 4090.

Two readings from the table deserve explicit note. First, the reward margin row is not a finding: by the DPO loss definition, margin equals β times the chosen-minus-rejected log-ratio, so a higher margin at higher β is a mathematical identity, not an alignment signal. The cross-beta quality metric is eval reward accuracy, which peaks at $\beta = 0.10$ (0.715) and is lower at both ends of the sweep. Second, entropy and mean token accuracy decrease monotonically as β falls,

Metric	$\beta = 0.05$	$\beta = 0.10$	$\beta = 0.20$	$\beta = 0.30$
Eval loss	0.655	0.658	0.640	0.639
Eval reward accuracy	0.701	**0.715**	0.694	0.694
Eval reward margin	1.169	1.443	1.793	2.093
Eval entropy	1.047	1.209	1.413	1.502
Eval mean token acc.	0.580	0.621	0.649	0.660
Wall-clock (min)	29.7	29.5	29.1	29.1

Table 6: DPO training-side metrics on the held-out preference validation set. Reward margin scales linearly with β by construction and is not a cross-beta quality indicator. Eval reward accuracy (bold at $\beta = 0.10$) is the comparable metric.

with $\beta = 0.05$ producing the most confident, most SFT-divergent policy, the precondition for the inference-time mode-collapse documented in Section 7. These training-side numbers measure alignment on training-distribution preferences; whether they predict deployment-side quality is the question Section 6 answers.

6 Comparative Evaluation

The evaluation compares six model states on a fixed set of twenty recruiter prompts: the untrained base SmolLM2-360M, the SFT model, and the four DPO-aligned variants at $\beta \in \{0.05, 0.10, 0.20, 0.30\}$. Three complementary instruments are used: automatic perplexity, LLM-as-judge pairwise win-rate with order-swap control, and behavioural surface statistics; followed by a synthesis.

6.1 Evaluation Set

The 20 prompts were fixed before any DPO training, eliminating the possibility of post-hoc cherry-picking. They split into two sub-sets of ten:

- **In-distribution (ind-01–ind-10).** Queries written in the style of the SFT training data but not drawn from it. These measure in-domain instruction quality.
- **Out-of-distribution (ood-01–ood-10).** Deliberately varied prompts: a junior boot-camp hire with no fixed stack, a principal staff engineer, niche stacks (Elixir/OTP, Rust/ WebAssembly, low-latency C++), a freelance documentation writer, a soft-skills-heavy role, and a remote-async senior Go engineer. These probe generalisation beyond the training distribution.

All six models are evaluated with greedy decoding and `repetition_penalty=1.3`, matching the Mini-Assignment 1 inference helper. The role of this parameter in preventing token-loop collapse is detailed in Section 7.

6.2 Perplexity

Perplexity is computed on the 750-example SFT validation set (the 250 held-out records, three queries each, formatted with the Section 2.3 template). It measures how well each model fits the instruction-following distribution, independent of any judge.

Model	Perplexity
Base (SmolLM2-360M)	11.66
SFT	**4.54**
DPO-β=0.30	4.80
DPO-β=0.20	5.03
DPO-β=0.10	6.01
DPO-β=0.05	8.84

Table 7: Perplexity on the SFT validation set (lower is better). Models sorted by perplexity. The base model is high because it has never seen the instruction template.

SFT sets the floor (4.54). DPO perplexity rises smoothly as β decreases from 0.30 to 0.10 (4.80 to 5.03 to 6.01), reflecting the expected alignment-fluency trade-off: lower beta allows the policy to diverge further from the SFT distribution. Below $\beta = 0.10$ a sharp cliff appears: DPO-β=0.05 reaches 8.84, a 47% jump over DPO-β=0.10. This is the first signal that $\beta = 0.05$ crosses the point where alignment gain no longer compensates for SFT-distribution divergence.

6.3 LLM-as-Judge Win-Rate

Pairwise win-rates are computed between every combination of the six model states. Six models yield 15 pairs; each pair is evaluated on all 20 prompts in both AB and BA orderings, giving 600 judge calls to Granite 3.3 2B via the `atlm_eval_judge` preset. An order-swap protocol filters position bias: only when both orderings agree is the pair-prompt counted as a consistent verdict; disagreement is recorded as inconsistency. Table 8 reports the results.

Three patterns stand out. First, the base model loses to every aligned model regardless of β: it produces no structured postings and the judge commits against it with 65-90% AB-BA agreement. Second, on the SFT vs. DPO pairs, DPO wins every consistent verdict across all four betas (0 SFT wins in 25 consistent pair-prompt verdicts), with zero losses. The signal is real but narrow given the low agreement rates on close pairs (20-45%), a judge-ceiling effect discussed in Section 7. Third, among the DPO-vs-DPO close pairs, the within-range betas sort lower-preferred (b01 > b02 > b03) when Granite commits, while DPO-b005 beats b02 and b03 but is effectively tied with b01 (2-1, 15% agreement).

6.4 Behavioural Statistics

Table 9 reports surface metrics across all 120 generated outputs (six models, 20 prompts each).

The base model produces no Markdown structure. SFT and the in-range DPO models produce outputs of comparable length (1,458-1,622 characters), consistently including `## Summary`

Comparison	A wins	B wins	Incons.	Agree.	Winner
Base vs. aligned					
Base vs. SFT	0	7	13	35%	SFT
Base vs. DPO-b005	2	13	5	75%	DPO-b005
Base vs. DPO-b01	0	18	2	90%	DPO-b01
Base vs. DPO-b02	1	14	5	75%	DPO-b02
Base vs. DPO-b03	1	12	7	65%	DPO-b03
SFT vs. DPO					
SFT vs. DPO-b005	2	7	11	45%	DPO-b005
SFT vs. DPO-b01	0	8	12	40%	DPO-b01
SFT vs. DPO-b02	0	6	14	30%	DPO-b02
SFT vs. DPO-b03	0	4	16	20%	DPO-b03
DPO vs. DPO (close pairs)					
DPO-b005 vs. DPO-b01	2	1	17	15%	—
DPO-b005 vs. DPO-b02	5	2	13	35%	DPO-b005
DPO-b005 vs. DPO-b03	5	1	14	30%	DPO-b005
DPO-b01 vs. DPO-b02	3	0	17	15%	DPO-b01
DPO-b01 vs. DPO-b03	5	1	14	30%	DPO-b01
DPO-b02 vs. DPO-b03	2	0	18	10%	DPO-b02

Table 8: Pairwise win-rate matrix. Consistent verdicts only (inconsistent pairs excluded from win counts). Agreement is the fraction of the 20 prompts where both orderings agreed. DPO-b005 vs. DPO-b01 is marked "—" as effectively tied (2-1, 85% inconsistency).

Model	Mean chars	Mean words	All 4 sections	Avg. sections
Base	694	89	0/20	0.00
SFT	1,567	208	3/20	2.80
DPO-b005	869	119	0/20	1.35
DPO-b01	1,458	196	1/20	2.50
DPO-b02	1,622	216	0/20	2.45
DPO-b03	1,592	211	3/20	2.85

Table 9: Behavioural statistics across all 20 evaluation prompts. "All 4 sections" counts how many of the 20 generations contained all four required Markdown headings. "Avg. sections" is the mean number of required headings present per generation.

and `## Required Skills`, but trailing off on `## Responsibilities` and `## Requirements`. DPO-b005 is the outlier: at a mean of 869 characters it is roughly 40% shorter than DPO-b01 and averages only 1.35 required sections per generation, worse than SFT on every structural metric. The entropy collapse visible in the training-side metrics (Table 6) manifests at inference as truncated, low-diversity outputs.

6.5 Synthesis

The three instruments agree on the headline ranking. Base loses to every aligned model. Among the aligned models, the in-range DPO betas (0.10, 0.20, 0.30) improve over SFT on judge-consistent verdicts (DPO wins 18 of 18 consistent pair-prompt comparisons across the three SFT-vs-DPO pairs, zero losses). On the perplexity axis the DPO models stay within a proportionate range of SFT (4.80-6.01), and their structural output statistics are comparable

to SFT. The $\beta = 0.05$ probe breaks this pattern on every axis: a 47% perplexity jump, a 40% drop in mean output length, and structural statistics worse than SFT, combined with a win-rate signal that becomes noise (DPO-b005 vs. DPO-b01 is 2-1 with 85% inconsistency). The alignment-quality peak sits at $\beta = 0.10$.

The deliverable policy is **DPO-b01** ($\beta = 0.10$, learning rate 5×10^{-5}): it beats the base 18-0 with 90% judge agreement, beats SFT 8-0 in consistent verdicts, and shows no fluency collapse or structural regression relative to SFT.

7 Limitations and Critical Discussion

Every finding in this section is grounded in a specific file or measurement produced during execution of Section 6; none is asserted from first principles.

7.1 Decoder-Loop Attractor and the Inference-Helper Regression

The first execution of Section 6 produced catastrophic token-level repetition collapses on several DPO outputs. The clearest case: DPO-β=0.20 on the cloud-engineer prompt `ind-07` emitted 42 consecutive repetitions of the token `SQS` as bullet items in the Required Skills section; DPO-β=0.30 on the same prompt emitted 29 consecutive `ECS` repetitions. These outputs actively confounded the evaluation judge: close-pair AB-BA agreement floored at 5-15%, well below the 50% expected from random and consistent with near-pure position bias in Granite's verdicts.

Two diagnostic findings followed that the training-side metrics could not have produced. First, the SFT corpus (`converted.jsonl`, 2,507 records) contains the phrase "Container orchestration using" exactly seven times, once per record at most. The failure is not memorised from training; it is emergent at inference under greedy decoding without a repetition penalty. Second, and more directly: the Mini-Assignment 1 inference helper `src/generate_mp1.py` carries `repetition_penalty=1.3` with the explicit comment *"small continued-pretrained models loop easily"*. The Mini-Assignment 2 evaluation generation function dropped this kwarg, silently, across notebook versions v3 through v6. Restoring `repetition_penalty=1.3` eliminated all token-loop collapses across the full 120-output evaluation run (six models, 20 prompts). Close-pair AB-BA agreement recovered to 15-40%, off the position-bias floor, and the DPO-over-SFT signal partially surfaced.

The methodological lesson is that inference-time decoder kwargs are load-bearing pipeline components, not afterthoughts. A parameter that solved the same failure mode on the same model family one assignment earlier was a silent regression in this one, producing apparent methodology-grade failures that training-side metrics could not detect. Cross-stage inference-kwarg parity is now a delivery checklist item.

7.2 Eval Judge Discrimination Ceiling

Granite 3.3 2B was selected from the calibration battery on the strength of a 10-call probe (5 pairs × 2 orderings). At production scale on 600 calls, a structural discrimination ceiling emerged that the calibration did not surface: on easy pairs (base vs. any aligned model) Granite produces 65-90% AB–BA agreement, but on close pairs (SFT vs. DPO, DPO vs. DPO) agreement falls to 15-40% even after the decoder-loop fix.

The arithmetic is informative. A purely random judge would produce 50% AB-BA agreement (independent coin flips). A purely position-biased judge would produce 0% (it always selects the candidate in a fixed slot; swapping the order swaps which model occupies that slot). The 15-40% close-pair regime lies between random and the pure-position-bias floor, consistent with partial signal recovery once catastrophic outputs were removed, but still far from the 65-90% Granite achieves on easy pairs. Inspection of raw verdicts revealed Granite occasionally identifying a response weakness in its reasoning text and then voting for that response anyway, suggesting the rubric application is at the edge of what a 2B-parameter model can do reliably on fine-grained comparisons.

The underlying cause is the calibration sample size. A 10-call probe is statistically incapable of distinguishing 80% AB-BA agreement from 50% at any meaningful confidence; only production-scale evaluation surfaces the floor. A close-pair-only calibration with $N \geq 30$ would have flagged the issue before the pipeline was built on it. This ceiling bounds the interpretability of the DPO-vs-DPO win-rate numbers in Table 8: the directional ordering (b01 > b02 > b03) is consistent but rests on small consistent-verdict counts.

7.3 DPO Coverage Bound

DPO can only suppress failure modes that appear in the candidate distribution used to build the preference dataset. The `ind-07` SFT defect (a tendency to fill the Required Skills section with near-duplicate "Container orchestration using X" bullets) is a prompt not included in the 250 preference prompts drawn in Section 4.1. DPO received no gradient signal pointing at this specific failure, which is why the defect persisted across all four DPO variants and surfaced as the token-loop catastrophes in the first evaluation execution. This is a concrete instance of a theoretical bound: DPO refines the policy in the regions where the candidate distribution and the judge's preferences both provide signal; outside those regions, SFT behaviour passes through unchanged.

7.4 Evaluation Set Size and Preference Volume

The 20-prompt evaluation set is sufficient for the headline base-versus-aligned gap and for detecting cross-beta directional differences when the judge commits, but binomial confidence intervals on close comparisons are wide. The 8-0 DPO sweep over SFT is robust at $N = 20$; the 3-0 b01-over-b02 close-pair count is suggestive rather than conclusive; the 2-1 b005-versus-b01

result is at the noise floor. Section 6.5's deliverable claim (b01 is the best-aligned model) rests on the robust end of these numbers; the within-range DPO ordering carries wider uncertainty.

The 6.5× scaling of preference triples (222 best-worst triples in Configurations B and C to 1,446 full-ranking triples in v6) did not break the over-fit signature: training reward accuracy reached 1.00 across all four betas in v6, equivalent to the near-saturation seen in prior configurations. It did not move the close-pair judge discrimination floor, and it did not produce a detectable deployment-quality lift. Preference-data volume was not the binding constraint for this setup. The bottleneck was preference-signal noise and the judge's discrimination ceiling.

7.5 Iteration History as a Methodological Finding

The pipeline reached its final form through four configurations, each motivated by the previous iteration's failure:

- **Configuration A** (Gemma judge): `max_tokens=800` caused 99% parse failures on the listwise ranking task; the judge's chain-of-thought alone exceeds 4,000 characters. Lifting to `max_tokens=16384` fixed it.
- **Configuration B** (Gemma judge, fixed): produced a decisive "DPO-b01 wins" narrative, but grounded in Gemma's own stylistic preferences: Gemma was the ETL teacher that shaped the SFT corpus, the RLAIF judge that ranked the preference candidates, and the evaluation judge that scored the result. Any Gemma prior compounded three times.
- **Configuration C** (cross-judge: Nemotron RLAIF, Granite eval): broke the shared-prior loop. The `ood-03` collapse that b01 exhibited under Configuration B: a 51-item repetition loop on an Elixir prompt, disappeared entirely under Configuration C, confirming it was a Gemma-judge artefact rather than a low-β intrinsic failure. Granite's close-pair discrimination ceiling surfaced.
- **v6** (consolidated four-beta, full-ranking, `repetition_penalty` restored): added the $\beta = 0.05$ cliff probe, scaled preferences to 1,446 triples, and fixed the decoder-loop regression that had contaminated all prior evaluation runs.

The full set of configuration artefacts is preserved on disk under suffixed output directories, making the trajectory reproducible independently of this report.

7.6 Future Work

Prioritised by expected impact:

1. **Larger evaluation judge.** A Gemma-class or Qwen3.5-9B-class model, or a judge ensemble, would raise the close-pair discrimination ceiling and make the DPO-vs-DPO ordering statistically interpretable. This is the single largest bound on what this evaluation could measure.

2. **Larger evaluation set.** At $N \geq 200$ stratified prompts, close-pair binomial confidence intervals shrink to the point where 3-0 or 5-1 counts become statistically meaningful.

3. **Full ($\beta\times$ LR) sensitivity grid with replication.** The current probe is one-dimensional along β at fixed LR. A grid sweep would map the interaction surface and quantify whether the b01 peak shifts at different learning rates.

4. **SFT corpus quality.** The `ind-07` defect is one concrete example of a patterned list shape the teacher model occasionally generates and the student memorises. Filtering or a higher-quality teacher would reduce inherited structural defects.

8 Conclusion

This report walked the full alignment pipeline for SmolLM2-360M on the IT job-postings domain, covering supervised fine-tuning, RLAIF-based preference dataset construction, and DPO training at four KL-coefficient values.

Supervised fine-tuning transformed the domain-adapted text completer from Mini-Assignment 1 into a structured instruction follower reaching a validation perplexity of 4.46 with only 2.34% of parameters trainable. DPO then refined that policy against preferences using a cross-judge protocol that keeps the RLAIF and evaluation roles assigned to model families distinct from one another and from the ETL teacher. The deliverable policy, DPO-β=0.10, defeats the base model on 18 of 18 consistent judge verdicts (90% AB-BA agreement) and sweeps SFT on every consistent close-pair verdict (8-0, zero SFT wins). A four-point beta sensitivity probe located the alignment-quality peak at $\beta = 0.10$: perplexity and win-rate both degrade below that value, with $\beta = 0.05$ producing a 47% perplexity cliff and a 40% drop in mean output length.

Three methodological lessons emerged from the iteration history. First, inference-time decoder kwargs are load-bearing: dropping `repetition_penalty=1.3` from the evaluation generation function, a parameter that solved the identical failure mode in Mini-Assignment 1, produced catastrophic token-loop collapses that the training-side metrics could not detect. Second, the calibration sample used to select the evaluation judge was undersized for the question it was asked: a 10-call probe cannot distinguish a 80% from a 50% AB-BA agreement rate, and production scale at 600 calls surfaced a structural discrimination ceiling on close pairs. Third, preference-data volume was not the bottleneck: a 6.5$\times$ scaling of preference triples did not move the over-fit signature, the judge floor, or the deployment-side win-rate; the binding constraint was preference-signal quality and judge capacity.

The aligned checkpoint, DPO-b01, is the input to the Final Project, where it will be integrated into a downstream recruiter-assistance system.

Bibliography

[1] Loubna Ben Allal et al. *SmolLM2: A Family of Small Language Models*. `https://huggingface.co/HuggingFaceTB/SmolLM2-360M`. 2024.

[2] Yuntao Bai et al. "Constitutional AI: Harmlessness from AI Feedback". In: *arXiv preprint arXiv:2212.08073* (2022).

[3] Edward J. Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: *arXiv preprint arXiv:2106.09685* (2022).

[4] Rafael Rafailov et al. "Direct Preference Optimization: Your Language Model is Secretly a Reward Model". In: *arXiv preprint arXiv:2305.18290* (2023).